Universidad Tecnológica de Hermosillo

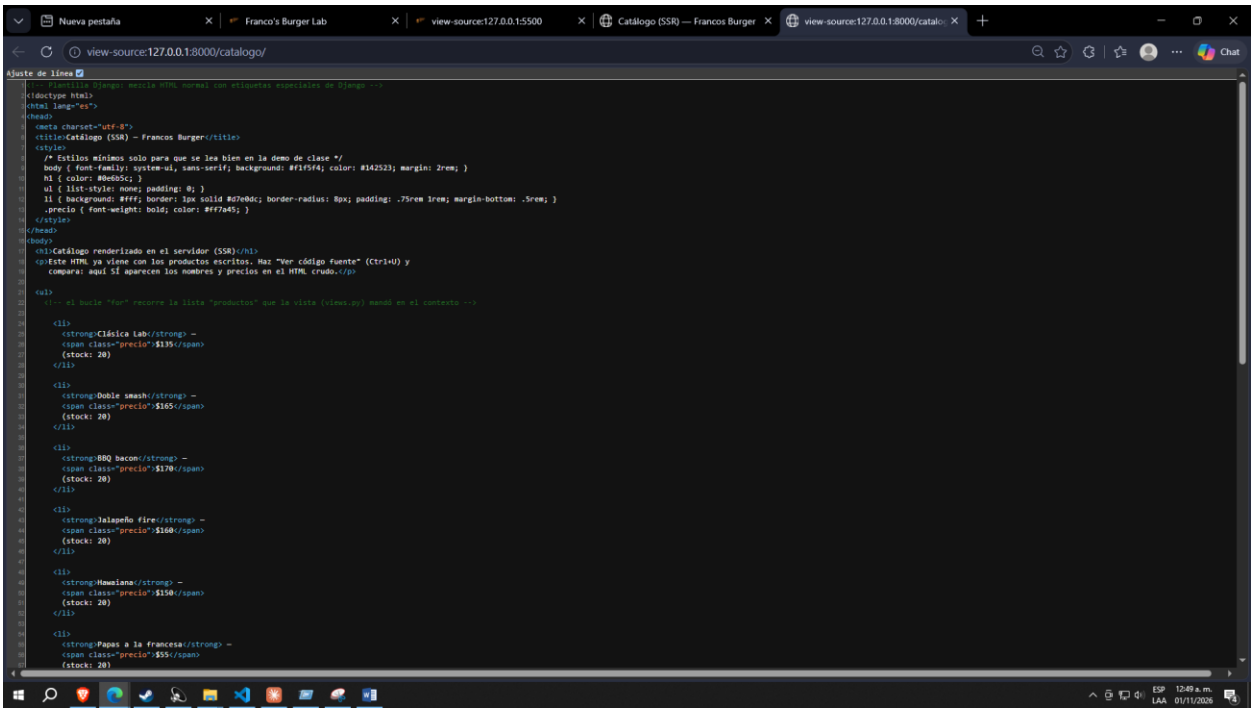
Evidencia 2 — Reporte de pruebas de la PWA (Unidad 2)

Franco's Burger Lab · Hamburguesería

Alumno	FRANCO GONZÁLEZ OMAR ANTONIO
Matrícula	22311070
Grupo	IDGS10-1
Asignatura	Aplicaciones Web Progresivas
Facilitador	Mtro. Prado Díaz Bernardo
Fecha	01/11/2026
Negocio · giro	Franco's Burger Lab · Hamburguesería
URL del repositorio	https://github.com/FrancoGonzalezOmarAntonio/francos-burger-lab-pwa_franco-gonzalez-omar-antonio
Hash del commit evaluado	c9f7c9e
URL de GitHub Pages	https://francogonzalezomarantonio.github.io/francos-burger-lab-pwa_franco-gonzalez-omar-antonio/

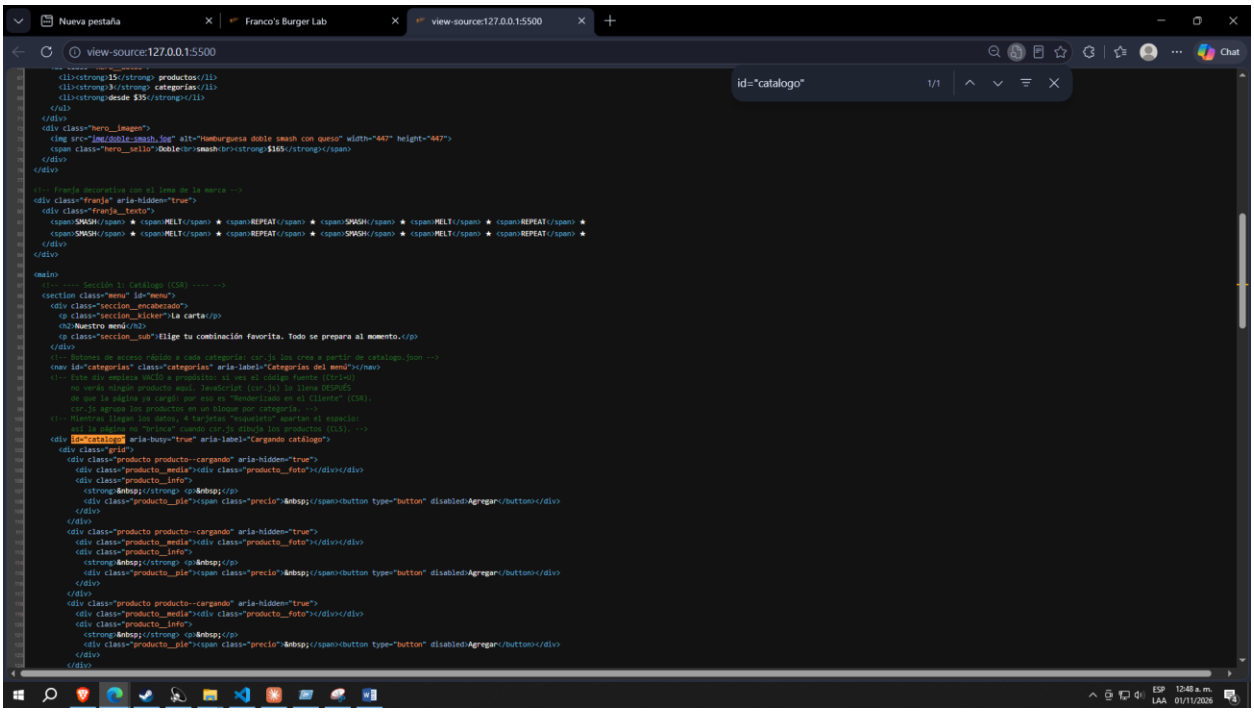
Prueba 1 — SSR: código fuente de Django

Con el servidor Django corriendo (python manage.py runserver), el código fuente de /catalogo/ ya trae los productos escritos dentro del HTML: el servidor arma la página completa antes de enviarla (Renderizado en el Servidor).



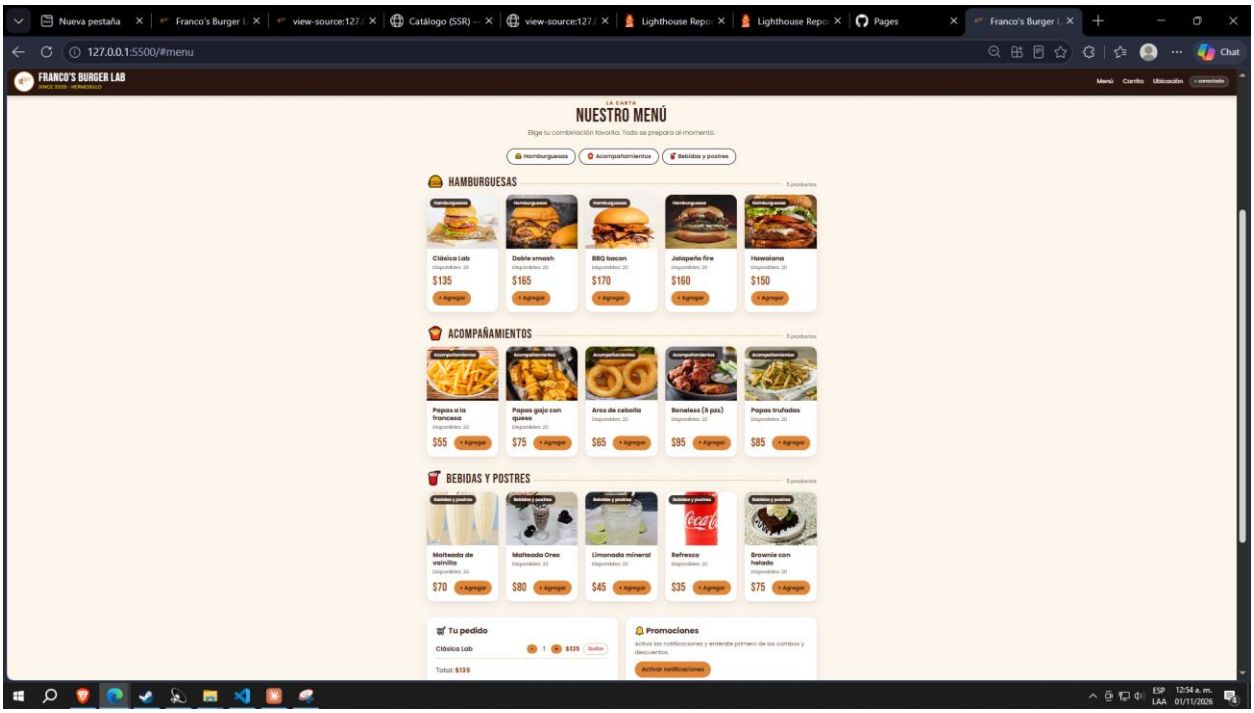
Prueba 2 — CSR: código fuente de index.html

En la versión PWA el llega vacío en el código fuente: los productos los dibuja JavaScript (csr.js) en el navegador después de leer catalogo.json (Renderizado en el Cliente). Contrasta con la prueba 1.



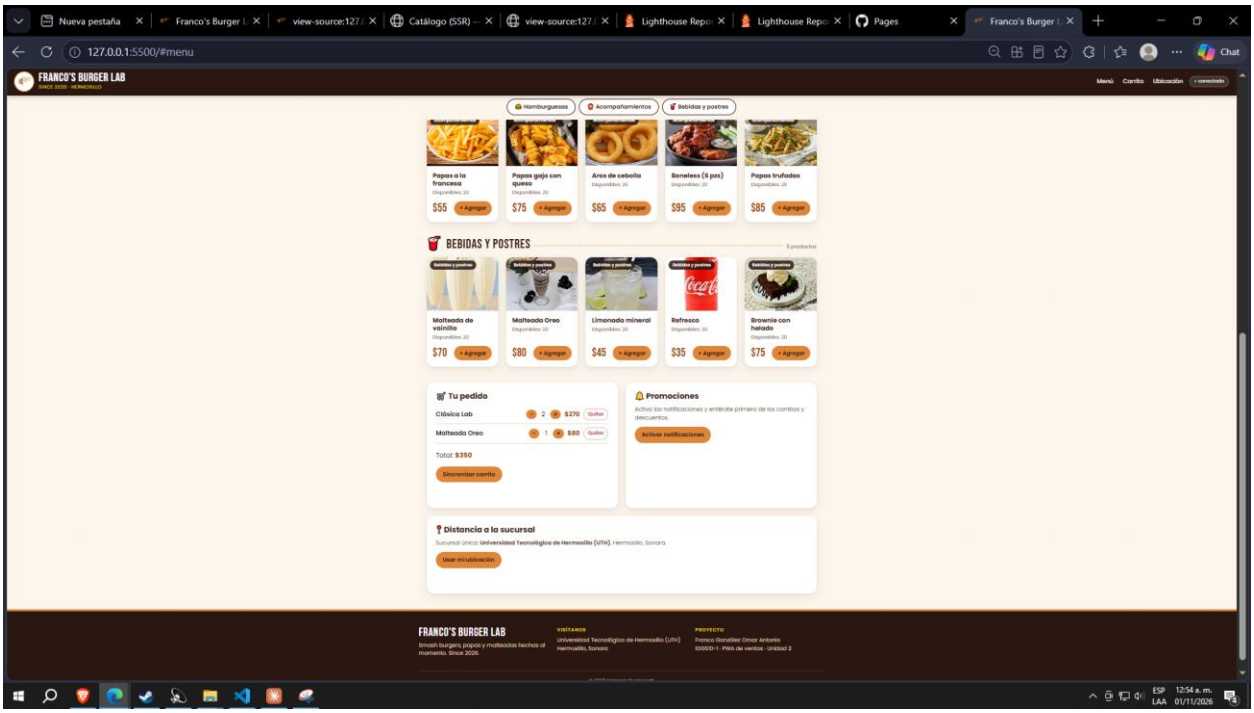
Prueba 3 — Catálogo con fotos

La app muestra el logo y los colores de Franco's Burger Lab y los productos del menú con su fotografía, precio y botón Agregar. El carrito ya tiene al menos un producto.



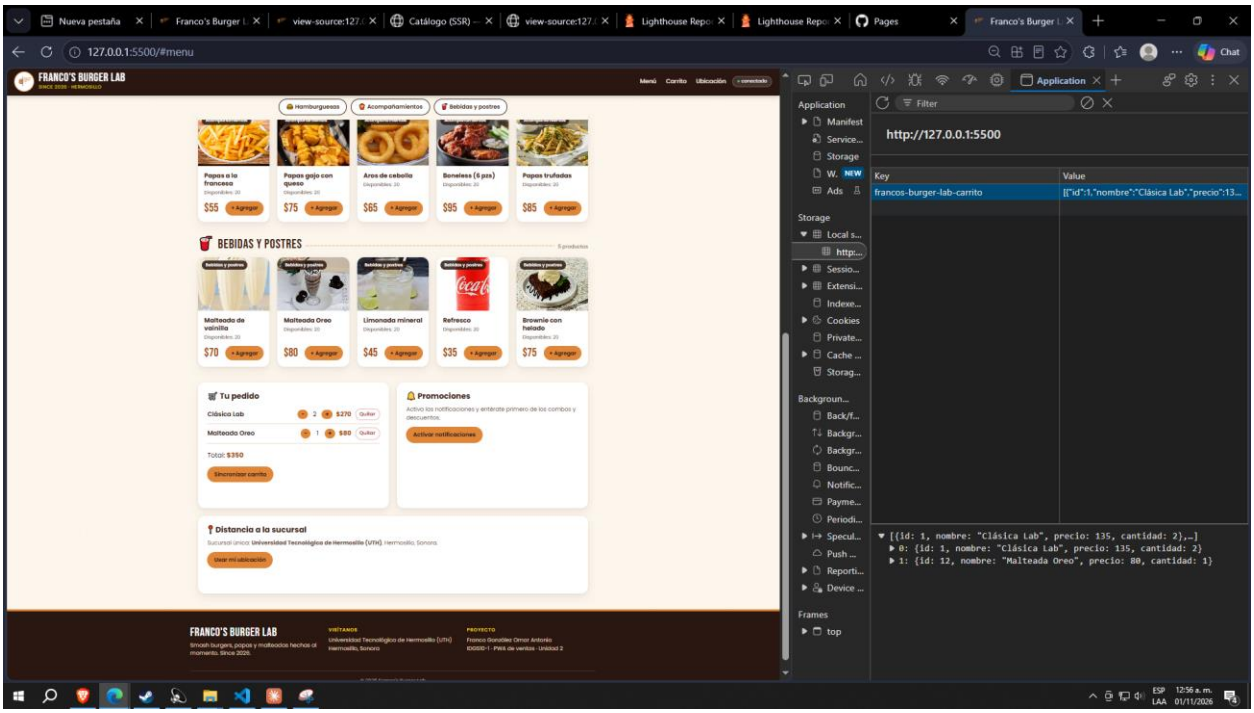
Prueba 4 — Carrito con botones + / −

Al subir la cantidad con el botón + el carrito recalcula automáticamente el subtotal del producto y el total del pedido.



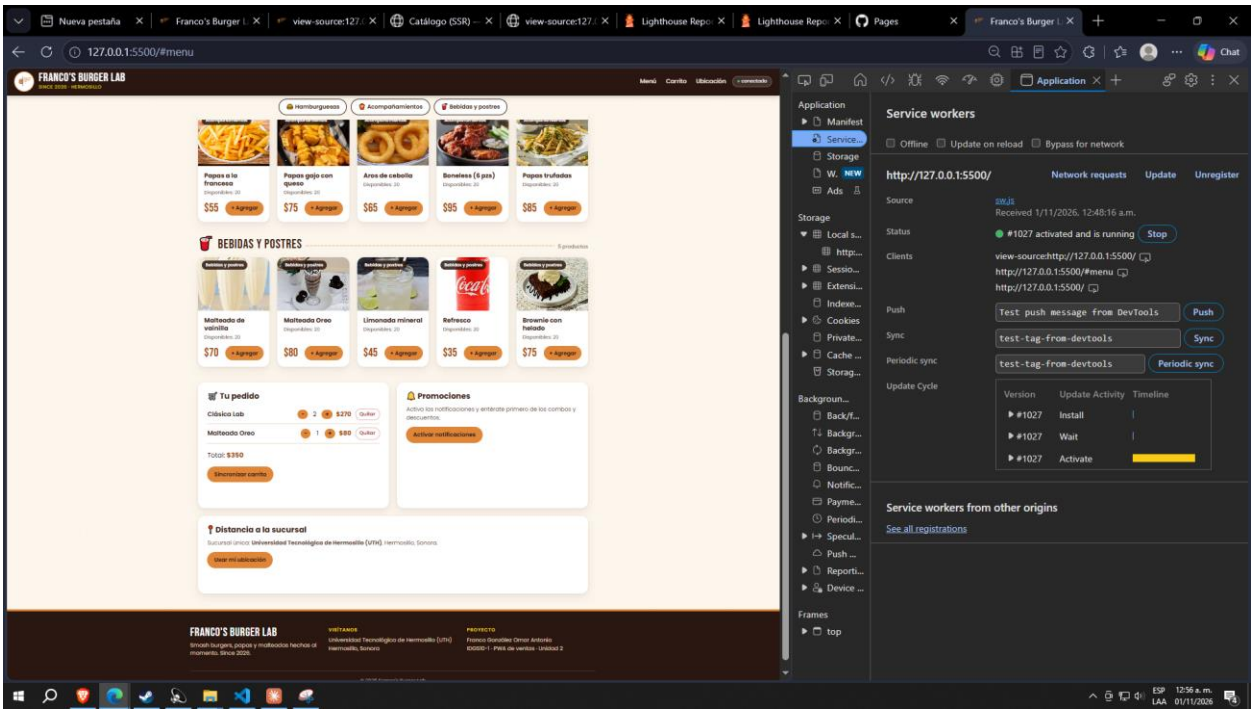
Prueba 5 — Carrito guardado en localStorage

El carrito se guarda en el localStorage del navegador como JSON, por eso sigue ahí al recargar o cerrar la app. Contiene los mismos productos y cantidades de la prueba 4.



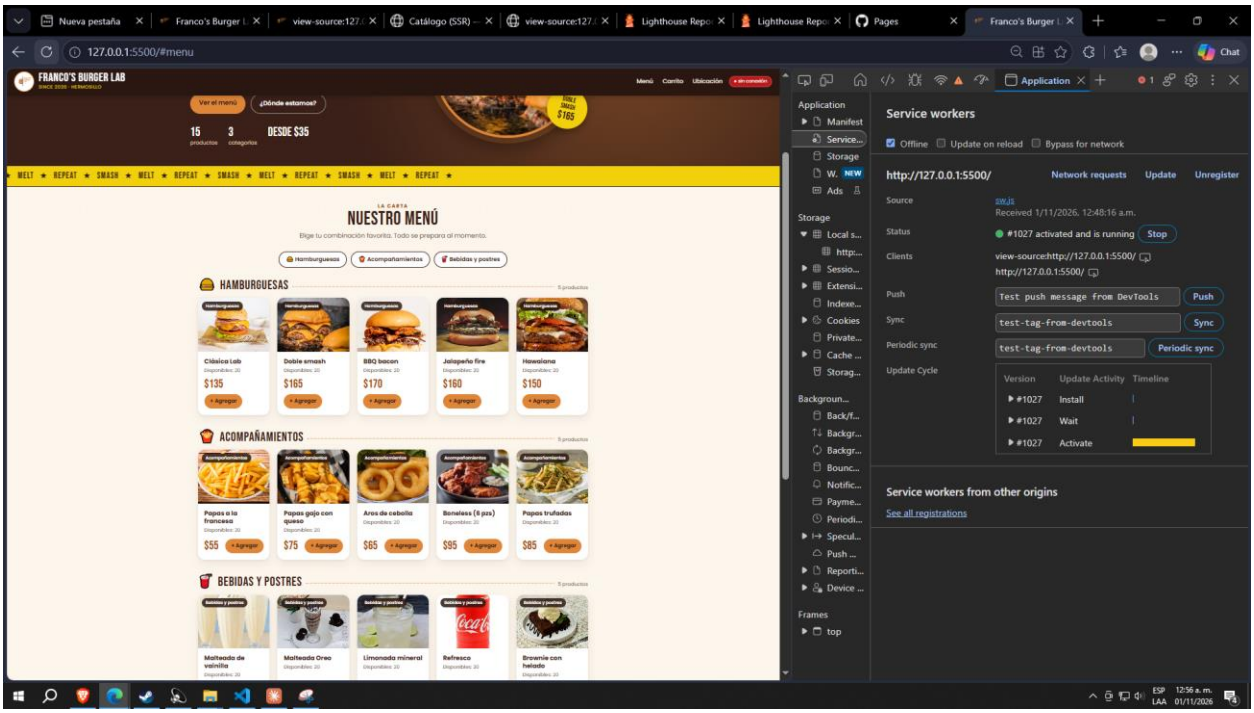
Prueba 6 — Service Worker activo

El Service Worker generado con Workbox (sw.js) está registrado y en ejecución: es el que guarda el App Shell, catalogo.json y las fotos en caché.



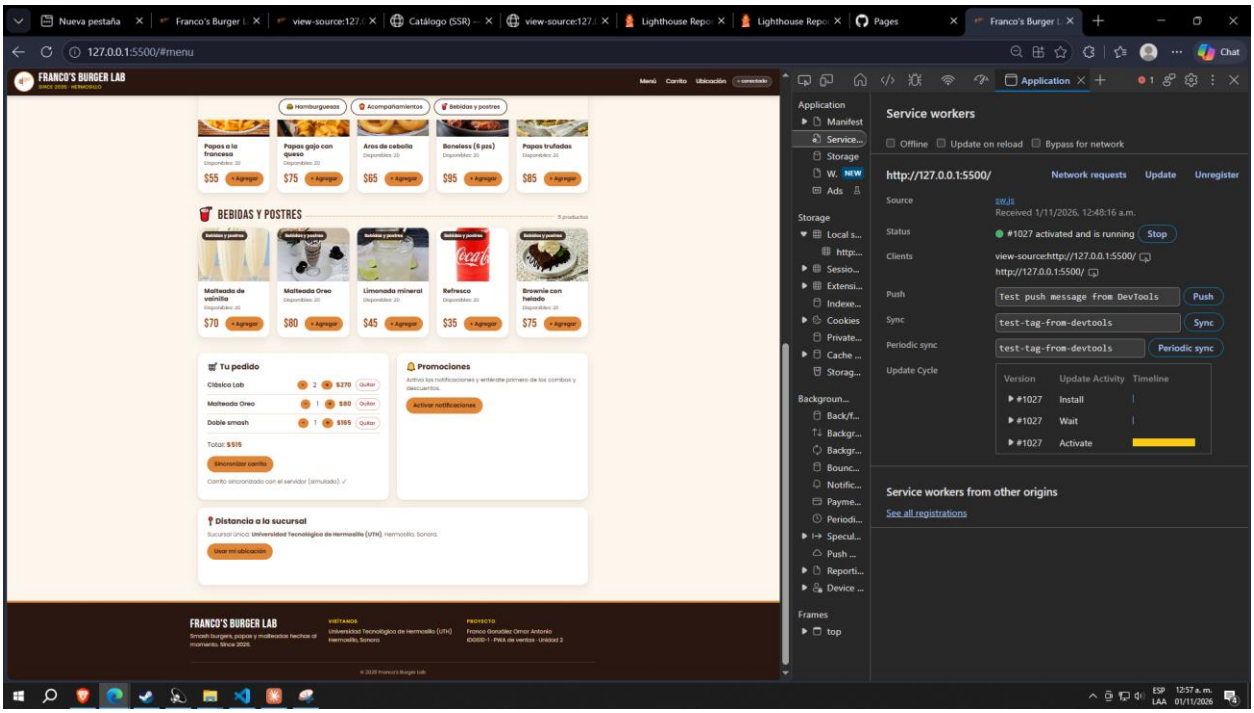
Prueba 7 — Funciona sin conexión

Con la red desactivada (Offline) y recargando la página, el catálogo y las fotos siguen apareciendo porque se sirven desde la caché del Service Worker. La píldora del encabezado cambia a "sin conexión".



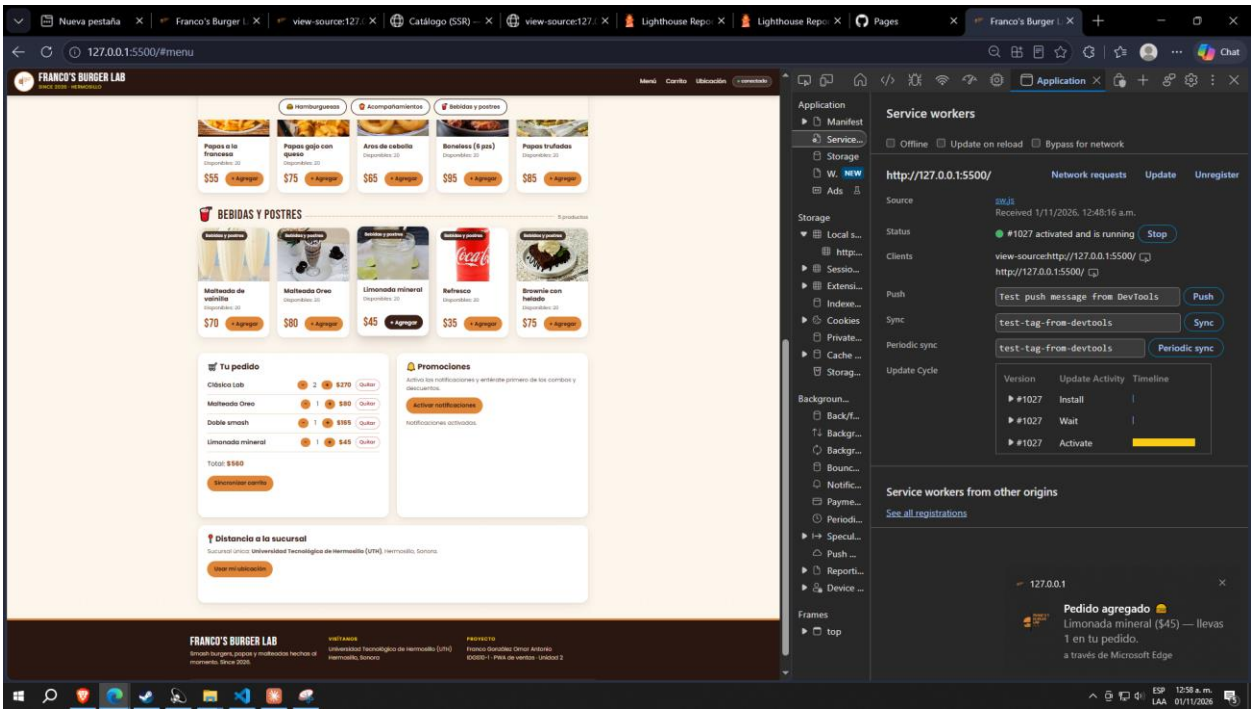
Prueba 8 — Sincronización del carrito

Un producto agregado sin conexión queda pendiente; al volver la conexión la app sincroniza el carrito y muestra el aviso de que se envió.



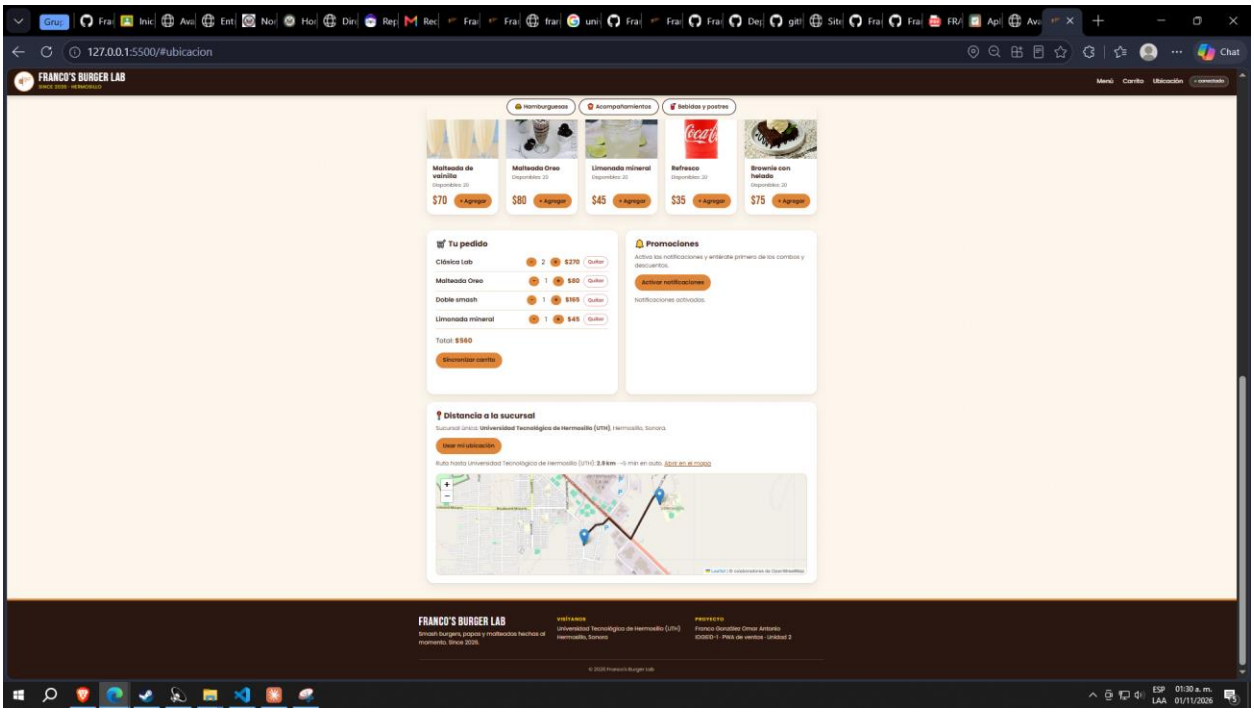
Prueba 9 — Notificación

Después de aceptar el permiso, al agregar un producto la app muestra una notificación del sistema con un texto de Franco's Burger Lab.



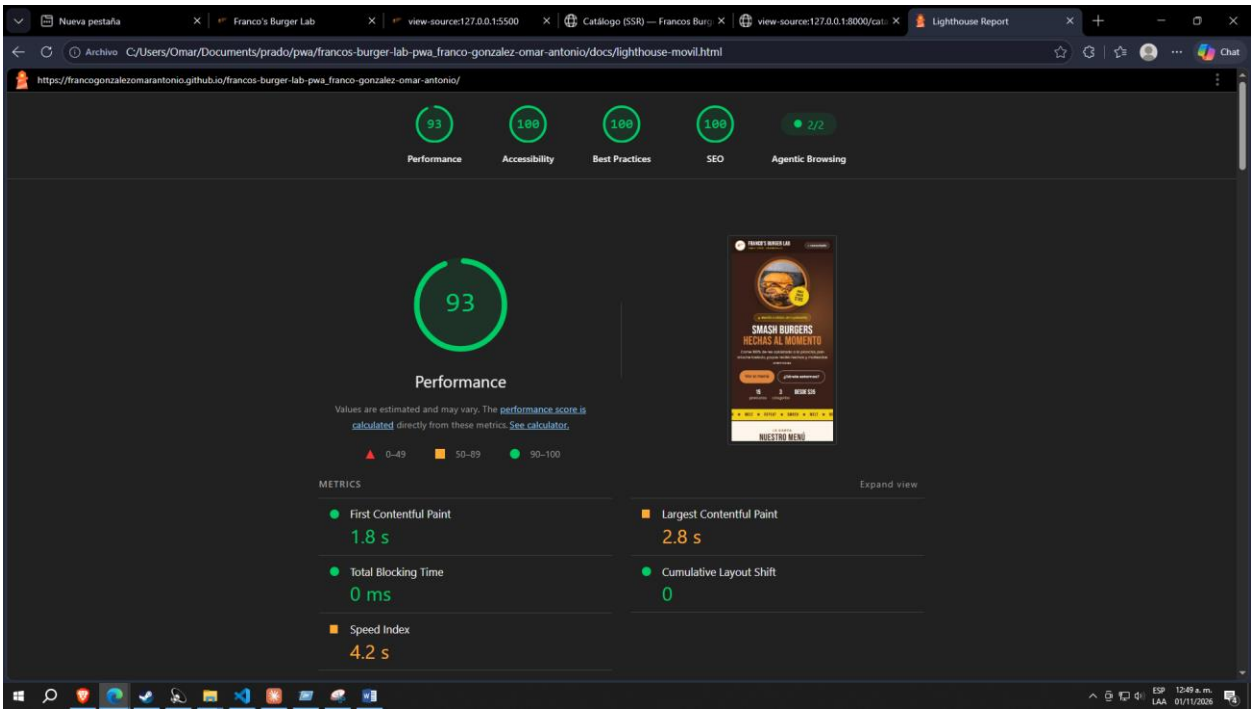
Prueba 10 — Ubicación y ruta a la sucursal

Con el permiso de ubicación aceptado, la app dibuja en el mapa la ruta por calles desde la ubicación del cliente hasta la sucursal de la Universidad Tecnológica de Hermosillo (UTH), con la distancia (2.9 km) y el tiempo estimado (~5 min en auto).



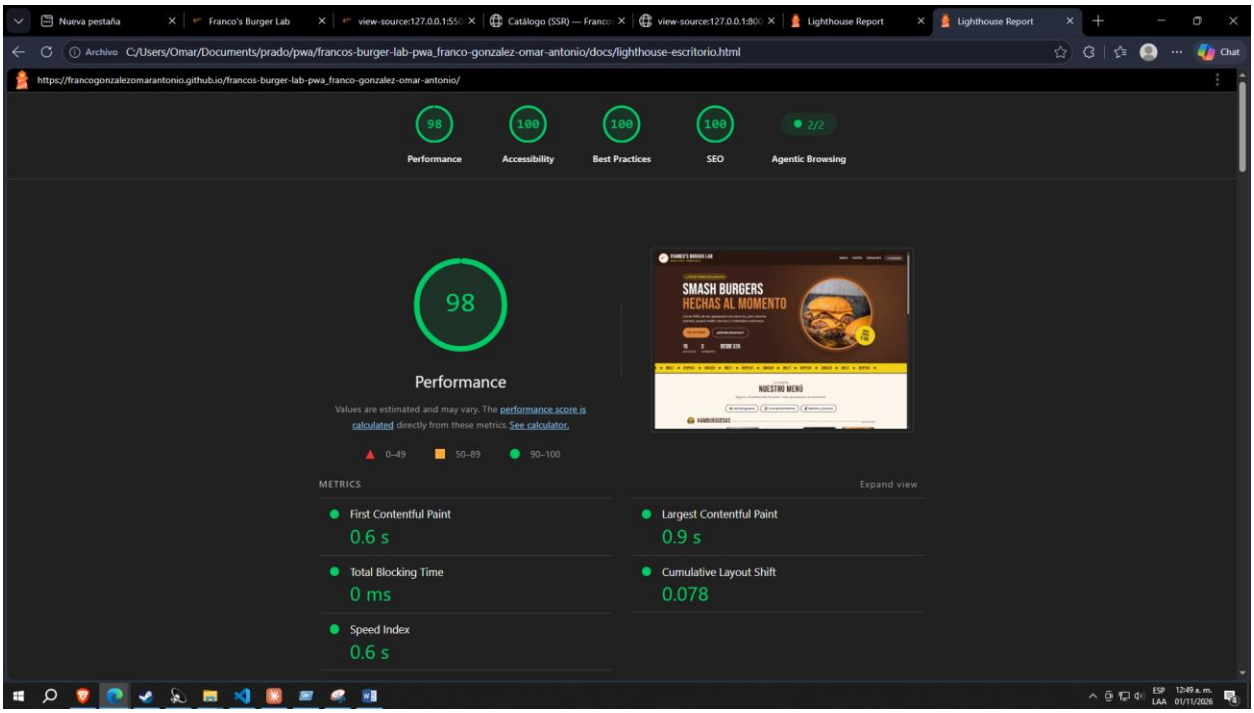
Prueba 11a — Lighthouse móvil

Auditoría de Lighthouse en modo móvil sobre la versión publicada en GitHub Pages (docs/lighthouse-movil.html): Performance 93, Accesibilidad 100, Buenas prácticas 100 y SEO 100.



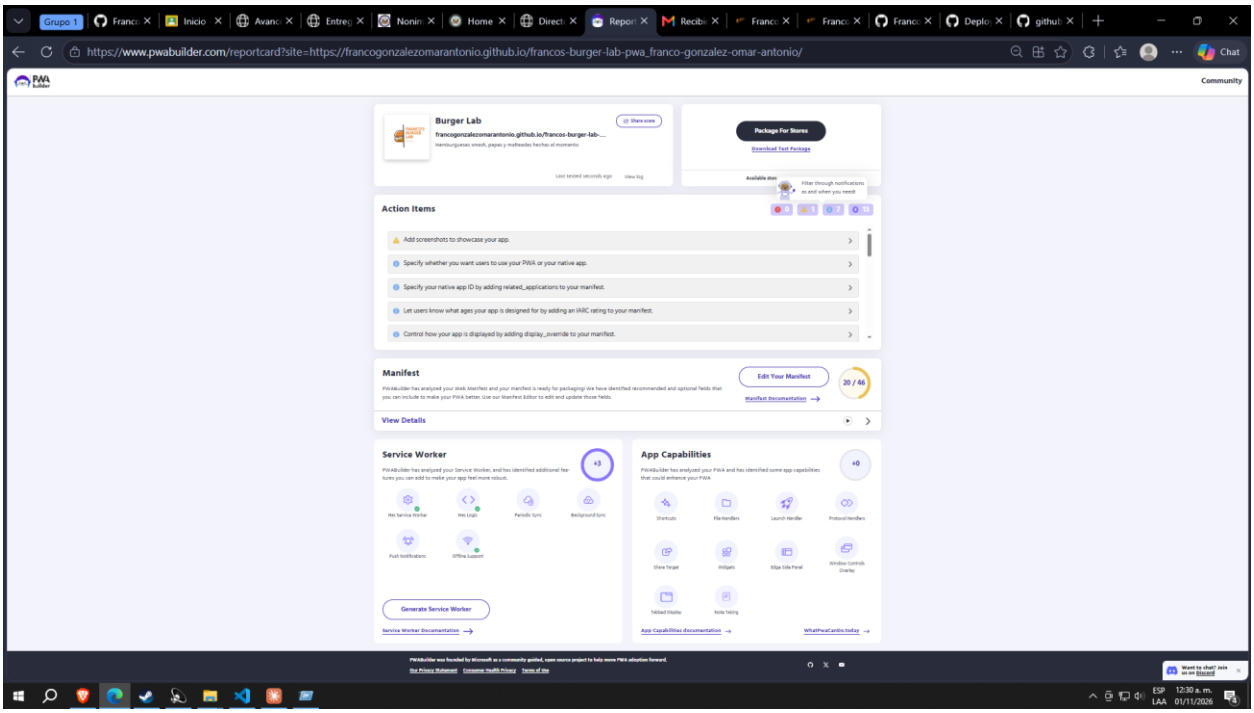
Prueba 11b — Lighthouse escritorio

Auditoría de Lighthouse en modo escritorio sobre GitHub Pages (docs/lighthouse-escritorio.html): Performance 98, Accesibilidad 100, Buenas prácticas 100 y SEO 100.



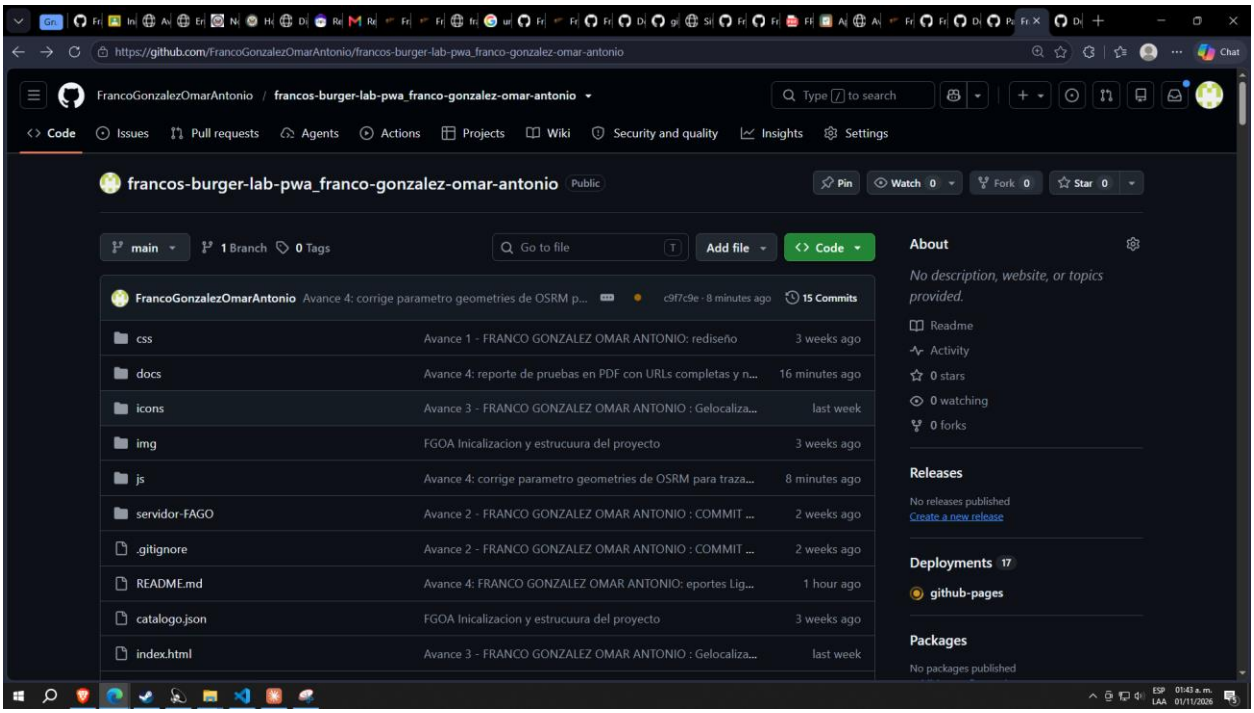
Prueba 12 — PWABuilder

PWABuilder analizó la URL de GitHub Pages: 0 errores rojos, el manifest está listo para empaquetar ("ready for packaging") y el Service Worker obtiene +3 (Has Service Worker, Has Logic y Offline Support).



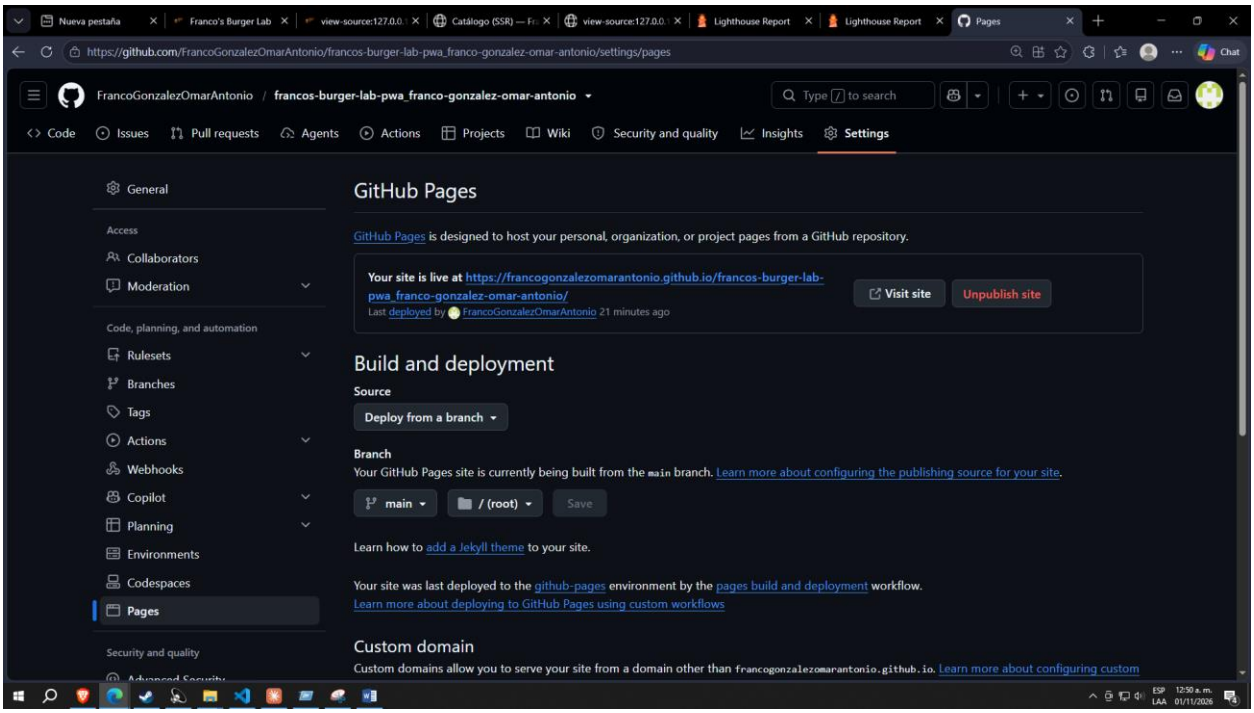
Prueba 13 — Repositorio en GitHub

Página principal del repositorio con la sesión iniciada. El hash del último commit es el mismo que aparece en la portada de este reporte.



Prueba 14 — GitHub Pages activo

GitHub Pages publica la PWA desde el repositorio; esa es la URL que se instala en el celular y la que auditan Lighthouse y PWABuilder.



Prueba 15 — App instalada en el celular

La PWA instalada en Android desde GitHub Pages y abierta desde su ícono: se muestra en pantalla completa, sin la barra del navegador (display: standalone), con el logo y el catálogo de Franco's Burger Lab.

